Machine Learning in Python

Lecture 4: Backpropagation and a neural network from scratch

Przemysław Rola

Department of Mathematics, Krakow University of Economics

Plan

1. The chain rule
2. Computation graphs and backpropagation
3. Looking back at the first example
4. A neural network from scratch

Based on F. Chollet, M. Watson, *Deep Learning with Python*, 3rd ed., Sec. 2.4–2.5.

The chain rule

Where we are

- Gradient descent needs, for every weight w, the derivative $\partial L/\partial w$ of the loss.
- Our network has 407 050 weights and the loss is a long chain of operations:

$$x \xrightarrow{\texttt{@ W1 + b1}} \xrightarrow{\texttt{relu}} \xrightarrow{\texttt{@ W2 + b2}} \xrightarrow{\texttt{softmax}} \xrightarrow{\texttt{cross-entropy}} L$$

- We need a systematic and cheap way to differentiate such chains.
- The tool: the **chain rule**; applied backwards through the network it is called **backpropagation**.

The chain rule

If $y = f(u)$ and $u = g(x)$, then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} \qquad \text{i.e.} \qquad (f \circ g)'(x) = f'\big(g(x)\big) \cdot g'(x).$$

Intuition: if u reacts 3 times as strongly as x, and y reacts 2 times as strongly as u, then y reacts $2 \cdot 3 = 6$ times as strongly as x.

Example

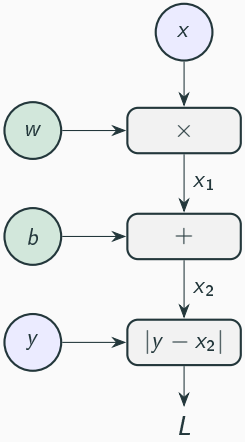
$y = (3x+1)^2$: $\quad u = 3x+1, \;\; y = u^2.$

$$\frac{dy}{dx} = \underbrace{2u}_{dy/du} \cdot \underbrace{3}_{du/dx} = 6(3x+1). \qquad \text{At } x = 1: \;\; 6 \cdot 4 = 24.$$

For longer chains: multiply the derivatives of all links.

Computation graphs and backpropagation

A computation graph



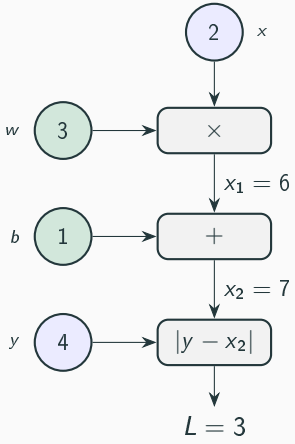
A tiny model with two parameters:

$$x_1 = w \cdot x$$
$$x_2 = x_1 + b \qquad \text{(prediction)}$$
$$L = |y - x_2| \qquad \text{(absolute error)}$$

- Each box is one simple operation.
- Green: parameters to learn; blue: data.
- Every model, also a neural network, can be drawn like this (just bigger).

(example from Chollet & Watson, Sec. 2.4.4)

Forward pass: compute the values

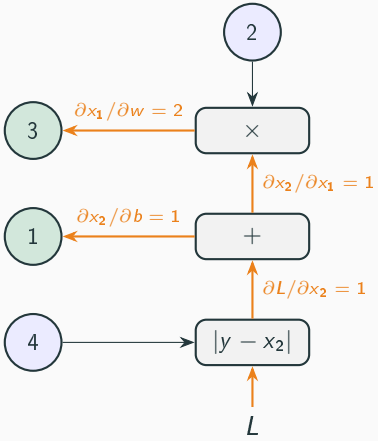


Data: $x = 2$, $y = 4$. Current parameters: $w = 3$, $b = 1$.

$$x_1 = 3 \cdot 2 = 6$$
$$x_2 = 6 + 1 = 7$$
$$L = |4 - 7| = 3$$

The **forward pass** goes from the inputs to the loss, and remembers all intermediate values.

Backward pass: derivatives of each link



Each link: "if the input of this box changes by ε, how much does its output change?"

- $L = |4 - x_2|$ and $x_2 = 7 > 4$: $L = x_2 - 4 \Rightarrow \partial L/\partial x_2 = 1$.
- $x_2 = x_1 + b \Rightarrow \partial x_2/\partial x_1 = 1, \ \partial x_2/\partial b = 1$.
- $x_1 = w \cdot x \Rightarrow \partial x_1/\partial w = x = 2$.

Chain rule along the path from L to each parameter:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial x_2} \cdot \frac{\partial x_2}{\partial x_1} \cdot \frac{\partial x_1}{\partial w} = 1 \cdot 1 \cdot 2 = 2,$$
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial x_2} \cdot \frac{\partial x_2}{\partial b} = 1 \cdot 1 = 1.$$

Check and use the result

Check numerically. Increase w from 3 to 3.01: $x_1 = 6.02$, $x_2 = 7.02$, $L = 3.02$.
The loss grew by $0.02 = 2 \times 0.01$ ✓ $(\partial L/\partial w = 2)$.

One step of gradient descent with $\eta = 0.1$:

$$w \leftarrow 3 - 0.1 \cdot 2 = 2.8, \qquad b \leftarrow 1 - 0.1 \cdot 1 = 0.9.$$

New prediction $x_2 = 2.8 \cdot 2 + 0.9 = 6.5$, new loss $|4 - 6.5| = 2.5 < 3$. ✓

Backpropagation

Go **backwards** from the loss, multiply the derivatives of the links, and reuse the products already computed (here $\partial L/\partial x_2$ is used for both w and b). In this way all gradients cost only a small multiple of one forward pass, even for millions of parameters.

Your turn

Same graph, but now $x = 2$, $y = 10$, $w = 3$, $b = 1$.

1. Forward pass: compute x_1, x_2, L.
2. Backward pass: compute $\partial L/\partial x_2$. *Careful: now* $x_2 < y$.
3. Compute $\partial L/\partial w$ and $\partial L/\partial b$.
4. Make one step of gradient descent with $\eta = 0.1$. Did the loss decrease?

Your turn

Same graph, but now $x = 2$, $y = 10$, $w = 3$, $b = 1$.

1. Forward pass: compute x_1, x_2, L.
2. Backward pass: compute $\partial L/\partial x_2$. *Careful: now* $x_2 < y$.
3. Compute $\partial L/\partial w$ and $\partial L/\partial b$.
4. Make one step of gradient descent with $\eta = 0.1$. Did the loss decrease?

Answers. $x_1 = 6$, $x_2 = 7$, $L = |10 - 7| = 3$. Since $x_2 < 10$: $L = 10 - x_2$, so $\partial L/\partial x_2 = -1$. $\partial L/\partial w = -1 \cdot 1 \cdot 2 = -2$, $\partial L/\partial b = -1$. New $w = 3.2$, $b = 1.1$: $x_2 = 7.5$, $L = 2.5$. ✓

Automatic differentiation

- Nobody computes gradients of real networks by hand.
- Frameworks (TensorFlow, PyTorch, JAX) record the operations of the forward pass as a computation graph and run backpropagation automatically: **automatic differentiation**.
- Keras 3 runs on top of any of the three ("backend").

TensorFlow: `tf.GradientTape`

PyTorch: `loss.backward()`; **JAX:** `jax.grad`

`GradientTape` in TensorFlow

```
import tensorflow as tf

w = tf.Variable(3.0)
b = tf.Variable(1.0)
x, y = 2.0, 4.0

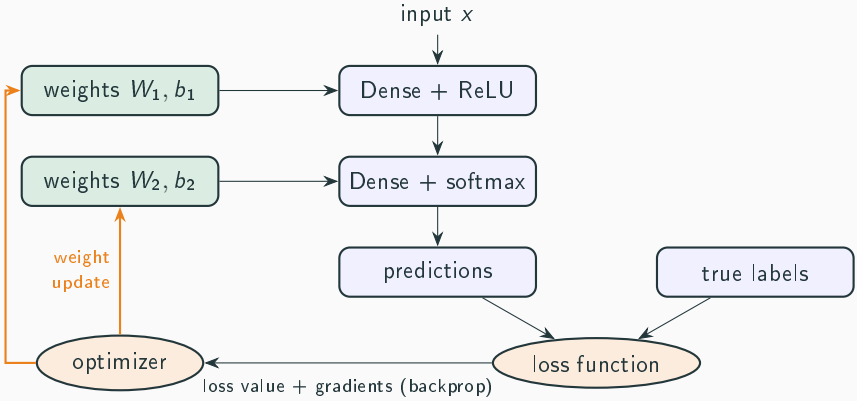
with tf.GradientTape() as tape:           # record the forward pass
    x2 = w * x + b
    loss = tf.abs(y - x2)

grad_w, grad_b = tape.gradient(loss, [w, b])
print(grad_w.numpy(), grad_b.numpy())     # 2.0 1.0   -- as computed by hand
```

The same works for tensors: `tape.gradient(loss, model.weights)` returns one gradient tensor for each weight tensor, with the same shape.

Looking back at the first example

The whole picture



`Sequential([...])` = the layers; `compile(loss=..., optimizer=...)` = the two ovals; `fit` = run this loop for every batch.

A neural network from scratch

Why rebuild what Keras already does?

- To see that there is no magic: a network is a few matrix products, an activation, a loss, and the gradient-descent update.
- We rebuild the MNIST model of Lecture 2 with low-level operations `keras.ops` (works with every backend) and `tf.GradientTape`.
- Building blocks:
  1. `NaiveDense`: one dense layer (weights `W`, `b`);
  2. `NaiveSequential`: a chain of layers;
  3. batches: consecutive slices of the data (as in Lecture 2);
  4. `one_training_step`: forward pass, loss, gradients, update;
  5. `fit`: loop over epochs and batches.

A dense layer

```
import keras
from keras import ops

class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation
        self.W = keras.Variable(shape=(input_size, output_size),
                                initializer="uniform")
        self.b = keras.Variable(shape=(output_size,),
                                initializer="zeros")

    def __call__(self, inputs):
        x = ops.matmul(inputs, self.W) + self.b
        if self.activation is not None:
            x = self.activation(x)
        return x

    @property
    def weights(self):
        return [self.W, self.b]
```

- `keras.Variable`: a tensor whose value can be changed (`assign`) — used for weights.
- `"uniform"`: small random initial weights (between -0.05 and 0.05); biases start at 0.
- `__call__`: exactly `activation(x @ W + b)` from Lecture 3.

A sequential model

```
class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs
        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def weights(self):
        weights = []
        for layer in self.layers:
            weights += layer.weights
        return weights
```

```
model = NaiveSequential([
    NaiveDense(28 * 28, 512, ops.relu),
    NaiveDense(512, 10, ops.softmax),
])
assert len(model.weights) == 4
```

The same model as `keras.Sequential([Dense(512, "relu"), Dense(10, "softmax")])`: four weight tensors W_1 (784, 512), b_1 (512,), W_2 (512, 10), b_2 (10,).

One training step

```
import tensorflow as tf
learning_rate = 1e-3

def update_weights(gradients, weights):
    for g, w in zip(gradients, weights):
        w.assign(w - g * learning_rate)                # gradient descent step

def one_training_step(model, images_batch, labels_batch):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)                                  # forward pass
        losses = ops.sparse_categorical_crossentropy(labels_batch, predictions)
        average_loss = ops.mean(losses)
    gradients = tape.gradient(average_loss, model.weights)                 # backward pass
    update_weights(gradients, model.weights)
    return average_loss
```

Exactly the five steps of the training loop from Lecture 3.

The training loop

```
def fit(model, images, labels, epochs, batch_size=128):
    for epoch in range(epochs):
        for start in range(0, len(images), batch_size):
            images_batch = images[start : start + batch_size]
            labels_batch = labels[start : start + batch_size]
            loss = one_training_step(model, images_batch, labels_batch)
        print(f"epoch {epoch}: loss of the last batch {float(loss):.2f}")
```

- Outer loop: epochs. Inner loop: batches of 128 consecutive images.
- The book wraps the slicing in a small `BatchGenerator` class; the idea is the same.
- `model.fit` in Keras additionally shuffles the data every epoch, reports metrics, ...

Evaluating the model

```
fit(model, train_images, train_labels, epochs=10, batch_size=128)

predictions = model(test_images)
predicted_labels = ops.argmax(predictions, axis=1)
accuracy = ops.mean(ops.cast(predicted_labels == test_labels, "float32"))
print(f"accuracy: {float(accuracy):.2f}")      # about 0.84 (book: 0.83)
```

- It works — but $\approx 83\%$ after 10 epochs, compared with $\approx 98\%$ for the Keras model after 5.
- The difference is the **optimizer**: plain SGD with a small learning rate 10^{-3} versus Adam. (The model is the same!)
- In the lab: try larger learning rates, and look at the loss after every epoch.

Summary

- **Chain rule**: derivative of a chain = product of the derivatives of the links.
- **Computation graph**: forward pass computes values; backward pass computes derivatives.
- **Backpropagation**: chain rule applied backwards with reuse of intermediate results; all gradients for the price of a few forward passes.
- Frameworks do it automatically (`GradientTape`, `backward`, `grad`).
- A whole MLP fits in ≈ 40 lines: dense layers, a loss, gradients, and the update $w \leftarrow w - \eta\, g$.

Lab: `Lab2_MLP_MNIST.ipynb`. **Next:** convolutional networks (CNN).

References

- F. Chollet, M. Watson (2025). *Deep Learning with Python*, 3rd ed. Manning. Sec. 2.4–2.5.
- D. E. Rumelhart, G. E. Hinton, R. J. Williams (1986). Learning representations by back-propagating errors. *Nature* 323, 533–536.
- A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research* 18(153), 1–43.